

ВВЕДЕНИЕ

В современных условиях эффективное управление личными финансами является важной задачей для достижения финансовой стабильности и реализации планов. Разрабатываемое веб-приложение «Денежная капсула» призвано стать удобным и надежным инструментом для решения этой задачи. Проект направлен на предоставление пользователям простого и наглядного механизма для повседневного контроля расходов и систематического накопления средств.

«Денежная капсула» — это веб-приложение для учета личных финансовых операций, которое позволяет не просто фиксировать доходы и расходы, а осознанно управлять своим бюджетом. С помощью приложения пользователь может регистрировать операции, распределяя их по категориям для лучшей аналитики, и устанавливать конкретные финансовые цели (например, накопление на крупную покупку). Для наглядности прогресс достижения целей отображается с помощью визуальных элементов.

Ключевым преимуществом приложения является ведение многовалютного учета. Для обеспечения точности расчетов реализована интеграция с банковским API для получения актуальных курсов валют в реальном времени, на основе которых выполняется автоматическая конвертация всех сумм. Это особенно полезно пользователям, имеющим доходы или совершающим траты в разных валютах.

Для обеспечения конфиденциальности реализована надежная система аутентификации, а данные надежно хранятся в структурированном виде. Интерфейс приложения интуитивно понятен и адаптирован для работы на различных устройствах, включая смартфоны и планшеты, а также поддерживает выбор темной темы для комфортной работы.

Важно отметить, что «Денежная капсула» фокусируется на базовых принципах учета и накопления. Приложение не предусматривает прямого доступа к банковским счетам пользователей или автоматической синхронизации транзакций. Также не реализованы функции инвестиционного планирования, совместного использования несколькими пользователями, формирования налоговой отчетности или прогнозирования будущих доходов.

1 ОБЗОР ИСТОЧНИКОВ ЛИТЕРАТУРЫ

В рамках данного проекта ставится важная задача разработки специализированного веб-приложения «Денежная капсула», предназначенного для учета личных финансов и накопления средств. В современных экономических условиях пользователям необходим надежный цифровой инструмент, позволяющий не только фиксировать доходы и расходы, но и осознанно управлять бюджетом, конвертировать валюты и отслеживать прогресс достижения финансовых целей.

Наличие удобного и функционального приложения позволяет систематизировать денежные потоки, избегать кассовых разрывов и повышать финансовую грамотность. При создании подобного продукта крайне важно разработать интуитивно понятный интерфейс, который обеспечит быстрый доступ к основным функциям: добавлению транзакций, настройке целей накопления и просмотру актуальных курсов валют.

В ходе функционирования программы предполагается работа с конфиденциальными данными пользователя, а также их структурированное отображение в виде списков операций и визуальных индикаторов прогресса. Это позволит получить всесторонний анализ финансового состояния.

Для успешной реализации проекта и проработки предметной области был изучен ряд фундаментальных источников, предоставляющих необходимую теоретическую и практическую основу как в сфере разработки, так и в сфере финансового планирования.

- Крейг Уолс – «Spring Boot in Action». В книге подробнорассматриваются принципы построения современных веб-приложений на базе фреймворка Spring Boot. Изучение этого источника позволило спроектировать надежную серверную архитектуру (REST API) и настроить взаимодействие с базой данных.

- Роберт Мартин – «Чистый код». Принципы, изложенные в книге, были использованы для организации структуры проекта, разделения логики на слои и написания поддерживаемого кода.

- Стив Круг – «Не заставляйте меня думать». Руководство по веб-юзабилити, позволившее спроектировать интерфейс, в котором пользователь может быстро внести транзакцию, не отвлекаясь на сложные элементы.

- Джордж С. Клейсон – «Самый богатый человек в Вавилоне». Классический труд по финансовой дисциплине. Описанный в книге принцип «заплати сначала себе» (откладывание фиксированной части дохода) лег в основу механики «виртуальной копилки» и системы приоритетности целей в приложении.

- Бодо Шефер – «Путь к финансовой свободе». В работе детально разбираются стратегии управления личным бюджетом и важность финансового планирования. Эти идеи нашли отражение в модуле постановки

целей, где пользователь обязательно задает конкретную сумму и срок, что повышает вероятность успешного накопления.

- Вики Робин – «Кошелек или жизнь». Книга помогла сформировать подход к осознанному потреблению, который реализован в приложении через систему категоризации расходов и наглядную визуализацию трат, побуждающую пользователя анализировать свои финансовые привычки.

Изучение данных источников позволило сформировать комплексный подход к разработке: техническая литература обеспечила надежность программного кода, а книги по финансовой грамотности помогли выстроить логику приложения так, чтобы оно реально помогало пользователю достигать финансовой стабильности.

1.1 Постановка задачи

В рамках данного проекта необходимо разработать веб-приложение «Денежная капсула», предназначенное для комплексного учета личных финансов, планирования накоплений и оперативной конвертации валют.

В современных экономических условиях пользователю важно иметь доступ к инструменту, который позволит не просто фиксировать траты, но и наглядно видеть прогресс движения к финансовым целям в мульти-валютной среде. Существующие решения часто перегружены лишним функционалом или не имеют гибкой привязки к актуальным курсам валют.

Задача приложения — предоставить удобный адаптивный интерфейс для регистрации пользователя, настройки целевых накоплений (с учетом дедлайна и валюты), ведения журнала доходов и расходов, а также автоматического пересчета сумм по текущим курсам банков.

Программа должна реализовывать сбор, обработку и отображение информации по следующим категориям:

- Профиль и безопасность — регистрация и аутентификация пользователей для защиты персональных финансовых данных.
- Финансовые цели — создание целей с указанием суммы, валюты (BYN, USD, EUR, RUB) и желаемой даты завершения; визуализация прогресса.
- Транзакции — ввод доходов и расходов с категоризацией, датой и описанием.
- Валютные операции — получение актуальных курсов валют в реальном времени (интеграция с внешним API) и предоставление инструмента «Конвертер» для быстрых расчетов.
- Персонализация — поддержка темной и светлой темы интерфейса для комфортной работы в любое время суток.

1.2 Обзор методов и алгоритмов решения поставленной задачи

Для реализации веб-приложения «Денежная капсула» выбрана классическая клиент-серверная архитектура, обеспечивающая надежность хранения данных и доступность сервиса с различных устройств.

Ключевым методом организации серверной части является использование архитектурного паттерна MVC (Model-View-Controller) в сочетании с принципами REST API. Это позволяет отделить бизнес-логику (расчет прогресса, обработку транзакций) от представления (HTML-страницы) и слоя хранения данных.

- Модульность: Логика приложения разбита на независимые контроллеры, что упрощает масштабирование и поддержку кода.
- Алгоритмы обработки данных
- Методы обеспечения безопасности: Применяется алгоритм хеширования паролей (BCrypt) перед сохранением в базу данных, что исключает утечку паролей в открытом виде. Аутентификация построена на механизме сессий, что является стандартом для веб-приложений.
- Визуализация и UX: На стороне клиента применяются методы динамического обновления DOM (Document Object Model). Вместо полной перезагрузки страницы используется технология AJAX (Fetch API) для отправки данных форм и получения обновленной статистики, что делает интерфейс отзывчивым и «плавным».

Таким образом, решение базируется на сочетании надежного серверного фреймворка, реляционной модели данных и реактивного клиентского интерфейса.

1.3 Выбор инструментов для реализации утилиты

Для успешной реализации приложения был произведен отбор инструментов разработки, обеспечивающих высокую производительность, безопасность и кроссплатформенность.

1. Язык программирования и платформа:

- Java 17 — в качестве основного языка бэкенда. Выбор обусловлен строгой типизацией, надежностью и развитой экосистемой.
- Spring Boot 3.2.0 — фреймворк, обеспечивающий быструю настройку веб-сервера (Tomcat), управление зависимостями и реализацию REST API. Использование модулей Spring Security и Spring Data JPA значительно ускорило разработку подсистем безопасности и доступа к данным

2. Система управления базами данных (СУБД):

- PostgreSQL — мощная объектно-реляционная СУБД. Выбрана за надежность, соответствие стандартам SQL и отличную поддержку целостности данных, что критически важно для финансового приложения.

3. Клиентская часть (Frontend):

- HTML5 / CSS3 — для разметки и стилизации адаптивного интерфейса.

- JavaScript (ES6+) — для реализации логики на стороне клиента (валидация форм, отправка запросов, управление анимациями). Использован «ванильный» JS без тяжелых фреймворков для обеспечения максимальной скорости загрузки страницы.

4. Сборка и управление зависимостями:

- Maven — инструмент для автоматической сборки проекта, управления библиотеками и зависимостями.

5. Внешние интерфейсы (API):

- API Беларусбанка — используется как источник достоверных данных о курсах валют в реальном времени.

Такой технологический стек (Java/Spring + PostgreSQL + JS) является индустриальным стандартом для создания корпоративных и частных веб-сервисов, гарантируя стабильность работы «Денежной капсулы» и простоту её дальнейшего сопровождения.

2 Требования пользователя

2.1 Программные интерфейсы

Система взаимодействует с внешними сервисами и состоит из внутренних модулей. Обмен данными между компонентами осуществляется через утвержденные интерфейсы с использованием стандартных протоколов.

2.1.1 Внешние программные интерфейсы

- Интеграция с банковскими API: Для получения актуальных курсов валют в реальном времени система взаимодействует с внешним API банка или финансового сервиса. Протокол взаимодействия — HTTPS/REST. Данные обменов защищаются с использованием TLS.
- Обработка ошибок соединения: Реализованы механизмы обработки сбоев при обращении к внешним API, включая повторные запросы с экспоненциальной задержкой и кеширование последних успешно полученных курсов на случай длительной недоступности сервиса.

2.1.2 Внутренние программные интерфейсы

- Система аутентификации: Для управления сессиями пользователей используется токен-ориентированная аутентификация. Токены передаются в заголовке каждого запроса для авторизации доступа к защищенным ресурсам.
- База данных: Для хранения всей структурированной информации системы используется СУБД PostgreSQL. Взаимодействие с базой данных осуществляется через драйвер с использованием языка SQL.
- Библиотеки визуализации: Для построения прогресс-баров, графиков и других элементов визуализации финансовых данных используются специализированные клиентские библиотеки.
- Взаимодействие фронтенда и бэкенда: Клиентская часть (веб-интерфейс) взаимодействует с серверной частью через API, построенное на принципах REST. Формат данных для обмена — JSON.

Описание программных интерфейсов составляет основу для обеспечения надежности, безопасности и масштабируемости системы. Четкое определение всех точек взаимодействия позволяет минимизировать риски интеграции, обеспечивает предсказуемость поведения системы под нагрузкой и формирует базу для будущего расширения функциональности. Данные решения соответствуют современным отраслевым стандартам, что гарантирует стабильность работы и упрощает дальнейшую разработку и поддержку приложения.

2.2 Интерфейс пользователя

Интерфейс системы представляет собой одностраничное веб-приложение (SPA) с адаптивным дизайном, обеспечивающим корректное отображение и удобство использования на персональных компьютерах, планшетах и смартфонах.

2.2.1 Структура и навигация

Основой интерфейса является главный экран, который содержит сводную информацию о финансовом состоянии пользователя. Навигация между ключевыми разделами осуществляется с помощью панели навигации, расположенной в нижней части экрана. Все операции по вводу данных (добавление доходов, расходов, целей) выполняются без перезагрузки страницы в разворачивающихся модальных окнах или блоках.

2.2.2 Организация и визуализация данных

Информация на главном экране организована в виде системы переставляемых и настраиваемых виджетов. Ключевые виджеты включают:

- Виджет финансовых целей: Отображает прогресс достижения целей с использованием прогресс-баров и графиков
- Виджет последних операций: Показывает историю доходов и расходов с возможностью сортировки и фильтрации.
- Виджет сводки: Отображает текущий баланс, общую сумму накоплений и основные финансовые показатели.

Визуализация данных реализована с использованием интерактивных элементов: прогресс-бары, диаграммы, а также символные элементы, такие как "интерактивная копилка", для наглядного отображения накоплений.

2.2.3 Уведомления и темы

Система предоставляет пользователю систему уведомлений, которые отображаются в интерфейсе. Уведомления информируют о статусе операций (успешное сохранение, ошибка), а также служат напоминаниями о приближающихся сроках достижения целей.

Для обеспечения комфорта работы при различных условиях освещения интерфейс поддерживает возможность выбора темы отображения: светлая и тёмная. Выбор темы сохраняется в настройках пользовательского сеанса.

2.2.4 Логика взаимодействия

Основные сценарии взаимодействия пользователя с системой приведены в таблице.

Таблица 1.2.1 — Сценарии взаимодействия пользователя с системой.

Действие пользователя	Реакция системы
Нажатие кнопки “Добавить цель”	Разворачивается форма ввода параметров цели (название, целевая сумма, срок).
Заполнение и отправка формы цели	Данные сохраняются в базе данных. В режиме реального времени обновляется виджет прогресса целей.
Нажатие кнопки "Добавить расход"	Открывается форма учета расходов с полями для суммы, категории, даты и комментария.
Ввод суммы расхода	Система в фоновом режиме рассчитывает и отображает влияние данной операции на сроки достижения всех активных целей.
Выбор валюты операции	Сумма автоматически конвертируется в основную валюту учета по актуальному курсу.
Просмотр главного экрана	Отображается актуальный прогресс по всем целям, список последних операций и текущие курсы валют.
Регистрация нового аккаунта	Создается уникальная учетная запись. Пользователь автоматически аутентифицируется, и для него создается сеанс.
Вход в существующий аккаунт	Производится аутентификация. Загружаются персональные данные пользователя, восстанавливается его сеанс и настройки (включая выбранную тему).
Добавление доходов	Обновляется баланс. Увеличивается доступная сумма для накоплений, корректируется прогресс по целям.
Изменение курсов валют	Производится автоматический пересчет всех сумм в основную валюту учета. Интерфейс обновляет отображаемые данные.

Интерфейс обеспечивает минималистичное и интуитивно понятное взаимодействие. Эргономичный дизайн и концентрация функциональности в рамках одной страницы приложения минимизируют временные затраты на освоение системы и повышают удобство ежедневного использования.

2.3 Характеристики пользователей

Данный раздел описывает целевую аудиторию приложения «Денежная капсула» для адаптации интерфейса и функциональности под их потребности и возможности. Определение портрета пользователя позволяет создать эффективный и интуитивно понятный продукт, соответствующий ожиданиям целевых групп.

Основные характеристики пользователей включают:

- Физические лица в возрасте от 14 лет
- Базовый уровень технической грамотности
- Опыт использования финансовых приложений не требуется
- Мотивация: контроль личных финансов и накопление средств
- Регулярность использования: от нескольких раз в неделю до ежедневного

Система ориентирована на решение базовых задач учета финансов и накопления средств без сложных аналитических функций. Интерфейс разработан с учетом минимального времени освоения и простоты ежедневного использования.

2.4 Предположения и зависимости

Для обеспечения стабильной и бесперебойной работы системы «Денежная капсула» необходимо учитывать комплекс критических факторов и условий, определяющих её функционирование, производительность и отказоустойчивость. Система разрабатывается как современное веб-приложение, что накладывает строгие требования к инфраструктуре, условиям эксплуатации и интеграции с внешними сервисами. Выполнение установленных технических и операционных требований является обязательным условием для корректной работы всех модулей приложения, включая системы хранения, обработки и визуализации финансовых данных.

Особое внимание следует уделять соблюдению современных технических стандартов и поддержанию стабильности работы всех компонентов системы - от серверной инфраструктуры до клиентских приложений. Необходимо учитывать, что система оперирует финансовыми данными пользователей, что требует реализации дополнительных мер безопасности и обеспечения надежности на всех уровнях архитектуры приложения. Это включает в себя не только технические аспекты, но и процедурные моменты, связанные с эксплуатацией и сопровождением системы.

Кроме того, важным аспектом является учет внешних зависимостей системы, включая работу с API сторонних сервисов и интеграционными решениями. Стабильность этих соединений напрямую влияет на функциональность приложения и качество предоставляемых пользователям услуг.

2.4.1 Технические зависимости

Фундаментальные технические требования к инфраструктуре системы включают:

- Стабильное интернет-соединение на стороне пользователя и сервера
- Исправность серверной инфраструктуры и базы данных PostgreSQL
- Поддержка современных стандартов HTML5, CSS3 и JavaScript браузерами

Нарушение любого из этих условий может привести к полной неработоспособности системы или существенному ухудшению качества обслуживания пользователей.

2.4.2 Внешние зависимости

Система зависит от следующих внешних факторов и сервисов:

- Доступность и стабильность работы внешнего банковского API курсов валют
- Соблюдение лимитов хранения данных и регулярное резервное копирование
- Обеспечение мер безопасности для защиты от несанкционированного доступа

2.4.3 Операционные предположения

Успешная эксплуатация системы предполагает выполнение следующих условий :

- Наличие у пользователя минимальных навыков работы с веб-интерфейсами
- Достоверность вводимых пользователем данных о финансовых операциях
- Соблюдение пользователем правил использования системы

Несоблюдение данных предположений может привести к некорректной работе системы и искажению финансовой аналитики.

Соблюдение всех перечисленных условий является обязательным для корректной работы приложения. Нарушение любого из указанных факторов может привести к частичной или полной неработоспособности системы, потере данных или некорректному выполнению финансовых расчетов.

3 Системные требования

3.1 Функциональные требования

Система должна обеспечивать комплексное решение для управления личными финансами с акцентом на удобство использования и наглядность представления данных. Основной функционал организован в виде логически связанных модулей, работающих в едином информационном пространстве.

Ключевые функциональные возможности системы включают:

- Регистрация новых пользователей с минимальным набором обязательных данных и аутентификация зарегистрированных пользователей с использованием защищенного механизма сессий
- Создание финансовых целей с указанием целевой суммы, временных рамок и приоритета выполнения, с возможностью последующей корректировки параметров
- Ввод и категоризация доходов и расходов пользователя с возможностью гибкой настройки системы категорий и тегов для детализации учета
- Отображение актуальных курсов валют с автоматическим обновлением и конвертация сумм между различными валютами в режиме реального времени
- Визуализация прогресса достижения финансовых целей с использованием интерактивных элементов интерфейса и расчет влияния финансовых операций на сроки достижения целей
- Отправка уведомлений о статусе целей и ключевых финансовых показателях через систему внутренних оповещений платформы

Все функциональные модули интегрированы в единую систему с обеспечением целостности данных и согласованности операций.

3.2 Нефункциональные требования

Система должна соответствовать современным стандартам качества программного обеспечения с акцентом на надежность, безопасность и производительность. Требования к эксплуатационным характеристикам сформулированы с учетом специфики веб-приложения для финансового учета.

Критически важные параметры системы включают:

- Надежность: время восстановления после сбоя не превышает 15 минут при полном исключении потери данных в штатном режиме работы
- Безопасность: аутентификация пользователей выполняется с использованием современного шифрования, пароли хранятся в хешированном виде с применением стойких алгоритмов

- Производительность: время отклика интерфейса не превышает 2 секунд при поддержке одновременной работы до 1000 пользователей
- Актуальность данных: обновление курсов валют происходит с частотой 1 раз в час с гарантией точности и достоверности информации
- Совместимость: система поддерживает работу в современных браузерах и на мобильных устройствах с iOS 12+ и Android 8+
- Удобство использования: освоение базового функционала занимает не более 30 минут благодаря интуитивно понятному интерфейсу

Достижение указанных показателей обеспечивает стабильную работу системы и удовлетворенность пользователей при ежедневной эксплуатации.

4 ТЕСТЫ ДЛЯ ПРИЛОЖЕНИЯ GYMTRACKER ULTIMATE

4.1. Юнит-тесты

На уровне модульного тестирования проверяется корректность работы отдельных компонентов бизнес-логики (Java-классы) и утилитарных функций.

Проверка валидации данных пользователя:

- Имя пользователя: не должно быть пустым.
- Пароль: длина не менее 4 символов.
- Уникальность: невозможность регистрации двух пользователей с одинаковым username.

Проверка логики финансовых операций:

- Сумма цели: должна быть строго больше 0.
- Конвертация валют: корректность расчёта кросс-курсов (например, USD -> EUR) на основе базовых курсов к BYN.
- Обработка транзакций: проверка присвоения правильного типа и даты

Проверка кеширования курсов валют:

- Проверка, что данные обновляются не чаще чем раз в 5 минут.
- Возврат дефолтных значений при недоступности API Беларусбанка.

4.2. Интеграционные тесты

Проверка взаимодействия между контроллерами, сервисами и базой данных PostgreSQL.

Регистрация и авторизация:

- Отправка POST-запроса на /api/auth/register → проверка появления записи в таблице users.
- Попытка входа с неверным паролем → получение ошибки 401/403.

Сохранение данных цели:

- Отправка JSON с параметрами цели → проверка сохранения в БД с привязкой к user_id.
- Проверка каскадного удаления: при удалении пользователя удаляются его цели и транзакции.

Взаимодействие с внешним API:

- Эмуляция ответа от сервера Беларусбанка и проверка корректного парсинга JSON/строки курсов.

4.3. UI/UX тесты

Проверка корректности отображения интерфейса и клиентской логики.

- Адаптивность: Проверка корректного скрытия SVG-изображения копилки на экранах шириной менее 900px и работоспособности кнопки меню на мобильных устройствах.
- Тёмная тема: Тестирование переключения тумблера корректного воспроизведения фонового видео и читаемости шрифтов в таблице валют.
- Визуализация: Проверка динамического изменения и корректности отображения процентов прогресс-бара

4.4. Функциональные тесты

Проверка ключевых сценариев использования приложения.

- Управление целью: Создание цели с валидацией даты и её удаление с полной очисткой истории транзакций.
- Финансовые операции: Увеличение накопленной суммы при добавлении дохода и её уменьшение при добавлении расхода.
- Валидация баланса: Проверка блокировки добавления расхода, если сумма операции превышает текущие накопления
- Конвертер: Проверка автоматического пересчета суммы при вводе значений и мгновенного обновления результата при смене валютной пары.

4.5. Нагрузочные тесты

- Проверка стабильности системы при одновременном обновлении курсов валют несколькими пользователями.
- Тестирование производительности списка транзакций при отображении истории из более чем 100 записей.

4.6 Тесты безопасности

- Защита API: Проверка невозможности вызова метода /api/data без валидной сессии.
- Санитизация данных: Проверка экранирования спецсимволов в полях «Описание» и «Название» для защиты от XSS-атак.
- Безопасность данных: Верификация того, что пароли в базе данных сохраняются исключительно в захешированном виде.

4.7. End-to-End тесты

Сквозной сценарий проверки полного цикла работы пользователя:

- Регистрация нового аккаунта и последующая успешная авторизация.
- Создание финансовой цели «Машина» в валюте USD.
- Внесение дохода в размере 100 USD и проверка визуального обновления прогресс-бара.

- Использование конвертера для пересчета накопленной суммы в RUB.
- Корректный выход из системы.

4.8. Регрессионные тесты

- Проверка сохранения активной сессии пользователя при фоновом обновлении кеша валют.
- Проверка восстановления выбранной темы оформления (светлая/темная) после перезагрузки страницы.

4.9. Тесты удобства (Usability)

- Понятность сообщений об ошибках.
- Логичность расположения кнопок.
- Минимальное количество действий для начала использования приложения.

5 СЦЕНАРИИ ВЗАИМОДЕЙСТВИЯ

5.1 Регистрация и вход в систему

- Пользователь вводит имя и пароль (с подтверждением) на форме регистрации.
- Система проверяет валидность данных и создает учетную запись.
- После редиректа пользователь выполняет вход.
- Система генерирует сессию и открывает главный экран.

5.2 Настройка финансовой цели

- Авторизованный пользователь открывает меню и выбирает создание цели.
- Заполняются поля: Название, Сумма, Валюта (BYN/USD/EUR/RUB) и Дедлайн.
- После сохранения данные записываются в БД, а на экране отображается заголовок цели и пустой прогресс-бар.

5.3 Добавление дохода

- Пользователь выбирает пункт «Добавить доход» в меню.
- Вводится сумма и описание операции.
- Система сохраняет транзакцию, обновляет баланс, визуальное заполняет «копилку» и добавляет запись в историю.

5.4 Добавление расхода

- Пользователь выбирает пункт «Добавить расход», вводит сумму и категорию.
- Система проверяет достаточность средств. Если сумма расхода превышает накопления, выдается ошибка.
- При успешной валидации сумма списывается, а прогресс достижения

5.5 Использование конвертера валют

Пользователю нужно быстро пересчитать сумму из одной валюты в другую. Основной поток:

- Пользователь нажимает кнопку конвертера.
- Открывается форма конвертера.
- Пользователь выбирает исходную валюту (например, USD) и целевую валюту (например, RUB).

- Вводит сумму в левое поле.
- Система мгновенно рассчитывает результат в правом поле, используя закешированные курсы валют.

5.6 Просмотр курсов валют

Пользователь хочет узнать актуальные курсы. Основной поток:

- На главном экране пользователь обращает внимание на виджет «Курсы валют Беларусбанка».
- Система автоматически (при загрузке страницы) опрашивает сервер.
- Если данные свежие (менее 5 минут), отображаются данные из кеша.
- Если данные устарели, сервер делает запрос к API банка и обновляет таблицу (Покупка/Продажа).

5.7 Удаление цели

Пользователь решил начать накопление заново или отменить текущую цель. Основной поток:

- Пользователь открывает меню и нажимает кнопку удаления.
- Появляется модальное окно подтверждения «Вы уверены? Вся история транзакций также будет очищена».
- Пользователь нажимает «Да».
- Система удаляет цель и все связанные транзакции из БД.
- Интерфейс сбрасывается к состоянию «Цель не установлена».